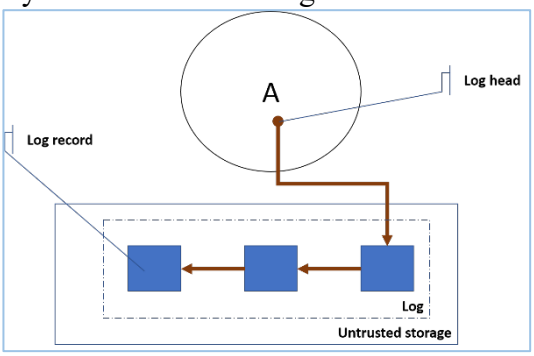
Foundations of Cybersecurity/Applied Cryptography
25 July 2023

EXERCISE N. 1 [ALL] **[12]**

1. Describe the RSA key generation algorithm.
2. Discuss the complexity of each step of the algorithm.

EXERCISE N. 2 [ALL] **[9]**

With reference to the figure, Alices wants to store an *event log* on an *untrusted storage*. A log is a list of blocks such that every block contains one or more events and the *log head* always points to the most recent event. For simplicity we assume here that every block contains a single event.



Alice requires that the log remains *tamper proof*, that is, although adversary can access the untrusted storage both in reading and writing mode, he must be not able to add, delete or modify events without being noticed.

The candidate is required to

1. Specify the format of the log block (e.g. using the using the C/C++ language).
2. Specify the format of the log head that is maintained in Alice's trusted storage.
3. Specify the implementation of the *append* operation which takes an event *e* and stores it into the log.

EXERCISE N. 3 [2022-23] **[9]**

Let us consider the RSA-2048 cipher ported on a 64-bit architecture[1]. Estimate the average cost of a decryption operation in terms of number of integer operations.

EXERCISE N. 3 [Earlier than 2022-23] **[9]**

Find, explain, and patch the possible vulnerabilities of the following function:

```
void do_stuff(int aux) {
  if (aux<0) {/*Handle error*/}
  unsigned char* buf = malloc(aux);
  if (gets(buf) == NULL) {
  /* Handle error */}
  /* do other stuff */
}
```

[1] This means that CPU registers are 64-bit.

Solution

EXERCISE N.1
See theory.

EXERCISE N.2
Answer 1. A block is composed of three fields: i) the *event* field; ii) the pointer to the *previous* block, and iii) the *hash* of the previous block. In C/C++ this could be

```
/* type definition */
typedef event …;
/* constant definition */
const uint HASHSIZE = … ;
/* Definition of global variables */
block*           headp = 0;
char[HASHSIZE]   headh = {0x00, 0x00,…, 0x00};

struct block {
     event          e;
     char[HASHSIZE] h;
     block*         p;
}
```

Answer 2. Alice must store the pointer to the head block (`headp`) and the hash of the head block (`headh`)

Answer 3. The append operation takes the following steps:

```
void append(event e) {
   block* b = (block *)malloc(block);
   b->e = e;
   b->p = headp;
   b->h = headh;
   headp = b;
   headh = hash(*b);
}
```

This scheme allows us to determine the first modified block.

EXERCISE N.3 [2022-23]
On average a decryption operation requires a number L of long operations with $L = 1.5 \times n$, where n is the number of bits of the modulus. In case $n = 2048$, the average number of long multiplications is $L = 1.5 \times 2048 = 3072$.

On a 64-bit architecture, a long multiplication requires $M = \left(\frac{2048}{64}\right)^2 = 1024$ integer multiplications and $R = \left(\frac{2048}{64}\right)^2 = 1024$ modulo reductions (same complexity as integer multiplications), for a total of $I = M + R = 2048$ integer operations (multiplications).
It follows that the average number ℓ of integer operations for a decryption is $\ell = L \times I = 3072 \times 2048 = 6.230.016$.

EXERCISE N.3 [Earlier than 2022-23]
The function contains the following vulnerabilities.

1) The code does not check whether `malloc` fails. Verbatim from *cppreference*: "On failure, returns a null pointer."
2) The `aux` parameter may assume value equal to `0`. In this case the behaviour of `malloc` is implementation-defined. A null pointer may be returned. Alternatively, a non-null pointer may be returned; but such a pointer should not be dereferenced and should be passed to `free` to avoid memory leaks."[2]
3) Function `gets()` is deprecated, because the programmer cannot specify the amount of bytes to transfer from the input stream to the buffer. Verbatim from *cppreference*: "The `gets()` function does not perform bounds checking, therefore this function is extremely vulnerable to buffer-overflow attacks. It cannot be used safely (unless the program runs in an environment which restricts what can appear on stdin). For this reason, the function has been deprecated in the third corrigendum to the C99 standard and removed altogether in the C11 standard."[3]
4) Memory allocated the `buf` buffer must be freed.

Consequently, a possible improvement of the function is the following:

```
void do_stuff(int aux) {
  if (aux <= 0){/*Handle Error*/;}
  unsigned char* buf = malloc(aux);
  if(buf == NULL){/*handle error*/}
  if (fgets(buf, aux, stdin) == NULL) {/* Handle error */}
  char* p = strchr(buf, '\n');
  if (p) { *p = '\0'; }
  /* do other stuff */
  free(buf);
}
```

N.B. Replacing character `'\n'` with the termination character `'\0'` depends on the application scenario. So, failing to do it is not a mistake *per se*.

[2] https://en.cppreference.com/w/c/memory/malloc
[3] https://en.cppreference.com/w/c/io/fgets